

Dokumentacja

Biometria

Projekt 2 - Rozpoznawanie człowieka na podstawie obrazu tęczówki

Bogumiła Okrojek

9 maja 2025

Spis treści

1	Opis aplikacji	2
1.1	Wykorzystane technologie i struktura	2
1.2	Interfejs użytkownika	2
1.3	Uzasadnienie wyboru technologii	3
2	Opis metod	4
2.1	Wczytanie i przygotowanie obrazu	4
2.2	Binaryzacja i obliczanie progów	4
2.3	Lokalizacja źrenicy	4
2.4	Wyznaczanie promieni źrenicy i tęczówki	5
2.5	Normalizacja tęczówki	5
2.6	Kodowanie tęczówki	6
2.7	Porównywanie kodów – odległość Hamminga	6
3	Wnioski	7
4	Podsumowanie	7

1 Opis aplikacji

Aplikacja została stworzona jako narzędzie do rozpoznawania osoby na podstawie analizy obrazu tęczówki oka. Jej głównym celem jest przeprowadzenie kompletnego procesu biometrycznego, obejmującego wykrycie źrenicy i tęczówki, normalizację, kodowanie przy użyciu falki Gabora oraz porównanie zakodowanych cech z wykorzystaniem odległości Hamminga.

1.1 Wykorzystane technologie i struktura

Aplikacja została zbudowana w języku Python z użyciem biblioteki **Tkinter**, która odpowiada za graficzny interfejs użytkownika (GUI). Wykorzystano również inne istotne biblioteki:

- **matplotlib** – do wizualizacji obrazów oraz prezentacji wyników poszczególnych etapów przetwarzania,
- **numpy** – do wykonywania operacji matematycznych i przetwarzania danych obrazu jako macierzy,
- **Pillow (PIL)** – do wczytywania i zapisu obrazów,

Struktura aplikacji podzielona została na następujące komponenty:

- **Interfejs graficzny (GUI)** – oparty na siatce **Frame** i elementach **Button**, **Label** oraz płótnach **FigureCanvasTkAgg** dla wyświetlania obrazów na różnych etapach przetwarzania,
- **Moduł funkcji przetwarzania obrazu** – zawiera funkcje odpowiedzialne za segmentację, normalizację, ekstrakcję cech i kodowanie obrazu tęczówki,
- **Moduł porównujący** – oblicza odległość Hamminga pomiędzy zakodowanymi obrazami w celu określenia podobieństwa.

1.2 Interfejs użytkownika

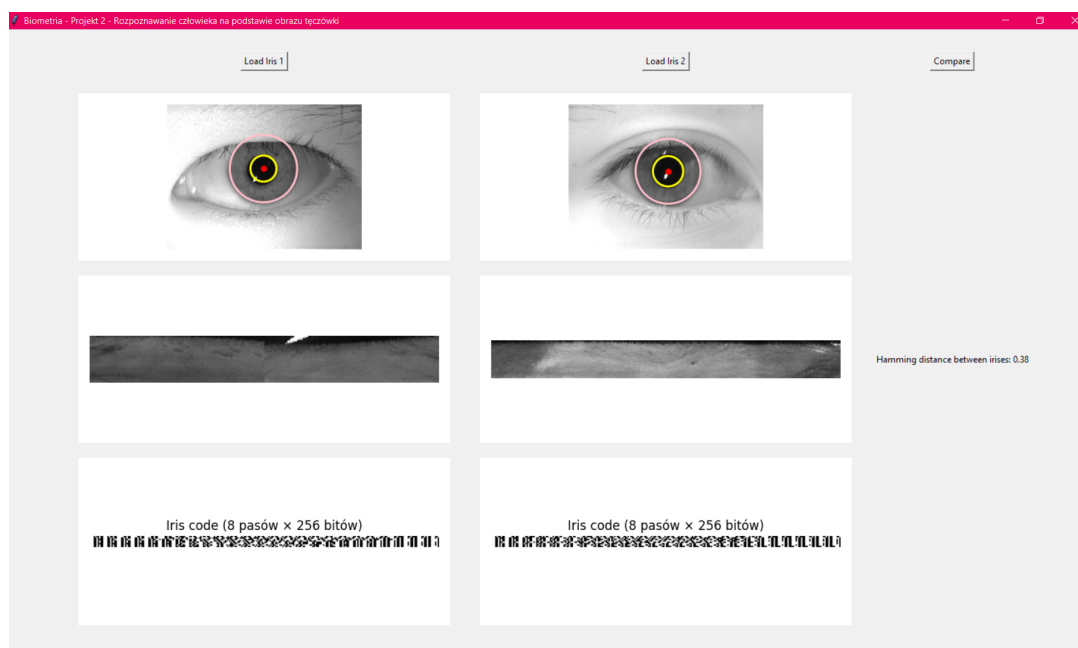
Interfejs został zaprojektowany w sposób przejrzysty i funkcjonalny. Użytkownik ma możliwość:

- Wczytania dwóch obrazów tęczówek do analizy za pomocą przycisków **Load Iris 1** oraz **Load Iris 2**,
- Automatycznego przetworzenia obrazu przez kolejne etapy: wykrycie środka źrenicy i tęczówki, ich promieni, normalizacja obrazu do postaci prostokątnej oraz kodowanie falką Gabora,

- Porównania wyników przy użyciu przycisku **Compare**, który oblicza odległość Hamminga pomiędzy dwoma zakodowanymi obrazami i wyświetla wynik w interfejsie.

Interfejs zawiera dwie kolumny, z których każda składa się z trzech wykresów:

1. Obraz z zaznaczonym środkiem i promieniami źrenicy oraz tęczówki,
2. Znormalizowany obraz tęczówki,
3. Wygenerowany binarny kod cech (tzw. *iris code*).



Rysunek 1: Widok interfejsu aplikacji do rozpoznawania tęczówki

1.3 Uzasadnienie wyboru technologii

Python został wybrany jako główny język implementacji ze względu na dostępność bogatego ekosystemu bibliotek naukowych i graficznych, a także prostotę integracji z GUI. **Tkinter** umożliwia szybkie prototypowanie aplikacji graficznych, natomiast **matplotlib** w połączeniu z **FigureCanvasTkAgg** pozwala na dynamiczne wyświetlanie wyników przetwarzania w czasie rzeczywistym.

Wybór falki Gabora jako metody ekstrakcji cech znormalizowanej tęczówki podyktowany był jej skutecznością w detekcji lokalnych wzorców teksturalnych, które są unikalne dla każdego oka. Porównanie zakodowanych cech za pomocą odległości Hamminga stanowi standardową metodę w biometrii tęczówki.

2 Opis metod

Celem niniejszej aplikacji jest realizacja procesu rozpoznawania osoby na podstawie unikalnych cech tęczówki oka. Metoda oparta jest na klasycznym podejściu do biometrii tęczówki, które obejmuje kilka kluczowych etapów: wstępne przetwarzanie obrazu, segmentację obszarów źrenicy i tęczówki, normalizację obrazu, ekstrakcję cech oraz kodowanie i porównywanie uzyskanych reprezentacji.

2.1 Wczytanie i przygotowanie obrazu

Pierwszym krokiem w procesie przetwarzania jest wczytanie obrazu za pomocą okna dialogowego. Użytkownik wybiera obraz w formacie PNG, JPG, JPEG lub BMP, który następnie jest konwertowany na macierz pikseli numerycznych. Tak przygotowany obraz stanowi bazę do dalszych operacji przetwarzania (`read_image`).

Aby ułatwić dalszą analizę, obraz jest następnie konwertowany do odcieni szarości za pomocą funkcji `change_grey`. Proces ten polega na obliczeniu średniej intensywności pikseli RGB (czerwony, zielony, niebieski) i przypisaniu tej wartości do pojedynczego kanału szarości. Dzięki tej operacji, redukujemy informację kolorystyczną, co upraszcza dalsze operacje, takie jak segmentacja tęczówki czy rozpoznawanie cech.

2.2 Binarizacja i obliczanie progów

Na podstawie średniej jasności obrazu (`countP`), obliczane są dwa progi: jeden dla tęczówki, a drugi dla źrenicy. Progi te są ustalane w oparciu o wartości domyślne (`pupil_cutoff` oraz `iris_cutoff`). Obliczając te wartości, funkcja `countP` oblicza średnią jasność obrazu i stosuje odpowiednie przeliczniki, które pomagają wyznaczyć wartości progowe dla obu obszarów.

Następnie obraz jest binaryzowany (`binary`) na podstawie obliczonych progów. Funkcja ta przekształca obraz na obraz binarny, gdzie piksele o intensywności wyższej lub równej progowi są ustawiane na 255 (biały), a pozostałe na 0 (czarny). Binarizacja umożliwia wyraźne oddzielenie obszaru źrenicy od tęczówki, co jest istotne w dalszych etapach analizy.

Cały proces obejmuje: wczytanie obrazu, konwersję do odcieni szarości, obliczenie progów dla źrenicy i tęczówki oraz binaryzację obu obszarów (`binary`).

2.3 Lokalizacja źrenicy

W celu zlokalizowania źrenicy na obrazie binarnym, wykonywane są następujące operacje: wypełnianie dziur (`fill_holes`), erozja (`erode_2d`) oraz obliczanie projekcji poziomych i pionowych,

które umożliwiają wyznaczenie środka źrenicy (`find_center`).

Proces rozpoczyna się od funkcji `fill_holes`, która wypełnia dziury w obrazie binarnym, poprawiając jakość maski źrenicy. Kolejnym krokiem jest zastosowanie operacji erozji (`erode_2d`), która pomaga usunąć małe, niepożądane fragmenty w obrazie, takie jak szumy.

Następnie obliczane są projekcje poziome (`projection_horizontal`) i pionowe (`projection_vertical`) obrazu, które sumują wartości pikseli wzdłuż odpowiednich osi. Wartości te są następnie wygładzane (`smooth_proj`), aby zminimalizować wpływ szumów i małych fluktuacji.

Na podstawie tych projekcji, funkcja `find_center` wyznacza współrzędne centrum źrenicy, obliczając środek masy na podstawie wygładzonych projekcji. W przypadku braku wyraźnych granic, współrzędne centrum są ustawiane na wartości zerowe.

Dzięki tym operacjom możliwe jest precyzyjne wyznaczenie położenia źrenicy, co jest kluczowe w dalszym przetwarzaniu obrazu.

2.4 Wyznaczanie promieni źrenicy i tęczówki

Na podstawie binarnych masek oraz wcześniej obliczonych współrzędnych środka, wyznaczane są promienie źrenicy i tęczówki (`find_rad`). W tym celu wykorzystywane są projekcje obrazu wzdłuż osi poziomej i pionowej, a także zmiany w tych projekcjach, które pozwalają określić granice źrenicy i tęczówki.

Proces wyznaczania promieni zaczyna się od odwrócenia binarnej maski (`inverse`) i wypełnienia dziur (`fill_holes`), a następnie stosuje się erozję (`erode_2d`) w celu usunięcia małych artefaktów. Kolejnym krokiem jest obliczenie sum projekcji poziomej (`projection_horizontal`) i pionowej (`projection_vertical`) obrazu, które są następnie wygładzane (`smooth_proj`) w celu zmniejszenia szumów.

Funkcja `find_changes` analizuje zmiany w tych projekcjach, identyfikując miejsca, gdzie następuje przejście od wartości 1 do 0 (co wskazuje na granice źrenicy i tęczówki). Na tej podstawie wyznaczane są promienie źrenicy i tęczówki. Finalnie, promienie te są uśredniane, a wynik stanowi promień tęczówki (`rad`).

Dzięki temu procesowi możliwe jest precyzyjne określenie promieni źrenicy i tęczówki, co jest niezbędne do dalszej analizy obrazu.

2.5 Normalizacja tęczówki

Zidentyfikowany fragment tęczówki został przekształcony do postaci prostokątnej, niezależnej od pozycji, rozmiaru źrenicy ani kąta spojrzenia. Proces ten polega na odwzorowaniu współrzędnych

biegunowych (promienia i kąta) na siatkę prostokątną.

Każdy piksel z przekształconego obrazu odpowiada punktowi na tęczówce o określonym promieniu (odległości od środka źrenicy) oraz kącie. Wysokość przekształconego obrazu odpowiada liczbie próbkowanych promieni pomiędzy promieniem źrenicy a tęczówki, natomiast szerokość — liczbie próbkowanych kątów. Dla każdego rzędu (stałego promienia) generowany jest indywidualny zakres kątów, tak aby długości łuków na kolejnych pierścieniach były zbliżone.

Zakresy kątów są wybierane symetrycznie względem osi pionowej przechodzącej przez środek źrenicy i obejmują dwa rozłączne przedziały: wokół kąta 0° (prawo i lewo) oraz wokół 180° (lewo i prawo w odbiciu). Szerokość tych przedziałów zależy od promienia danego pierścienia i maleje w miarę oddalania się od źrenicy, aby zapewnić zbliżoną długość odczytywanych wycinków.

2.6 Kodowanie tęczówki

Znormalizowany obraz tęczówki jest dzielony na osiem poziomych pasów o równej wysokości. Każdy pas jest analizowany niezależnie w celu ekstrakcji cech opisujących strukturę tęczówki.

Dla każdego pasa wybieranych jest 128 równomiernie rozłożonych punktów wzdłuż szerokości obrazu. W każdym punkcie wyznaczana jest średnia intensywność na podstawie kolumny pikseli, z wykorzystaniem wag okna Gaussa. Uzyskany w ten sposób wektor cech 1D reprezentuje strukturę danego pasa.

Na tej reprezentacji wykonywana jest filtracja zespoloną falką Gabora, która wydobywa informacje o częstotliwościach przestrzennych w danych lokalizacjach. Odpowiedź filtru jest następnie kwantyzowana do jednej z czterech ćwiartek płaszczyzny zespolonej. Każdej ćwiartce przypisuje się dwubitowy kod binarny, co pozwala uzyskać 256-bitowy ciąg znaków dla każdego pasa.

Rezultatem działania funkcji `koduj_iris` jest więc binarna matryca cech o wymiarach 8×256 , stanowiąca unikalny podpis danej tęczówki.

2.7 Porównywanie kodów – odległość Hamminga

W celu porównania dwóch zakodowanych wzorców tęczówki zastosowano metrykę Hamminga, zaimplementowaną w funkcji `hamming_distance`. Odległość Hamminga mierzy odsetek bitów różniących się między dwoma binarnymi ciągami o takiej samej długości. W praktyce oznacza to stopień niezgodności między dwoma kodami – im mniejsza wartość, tym większe podobieństwo między tęczówkami.

Dla dwóch macierzy binarnych reprezentujących kody (8×256), porównanie realizowane jest przez bezpośrednie zliczenie różniących się bitów i podzielenie ich przez całkowitą liczbę bitów.

Wynik odległości Hamminga jest następnie prezentowany użytkownikowi za pomocą inter-

fejsu graficznego w funkcji `compare`, która aktualizuje odpowiedni element GUI informacją o stopniu podobieństwa między dwoma kodami.

3 Wnioski



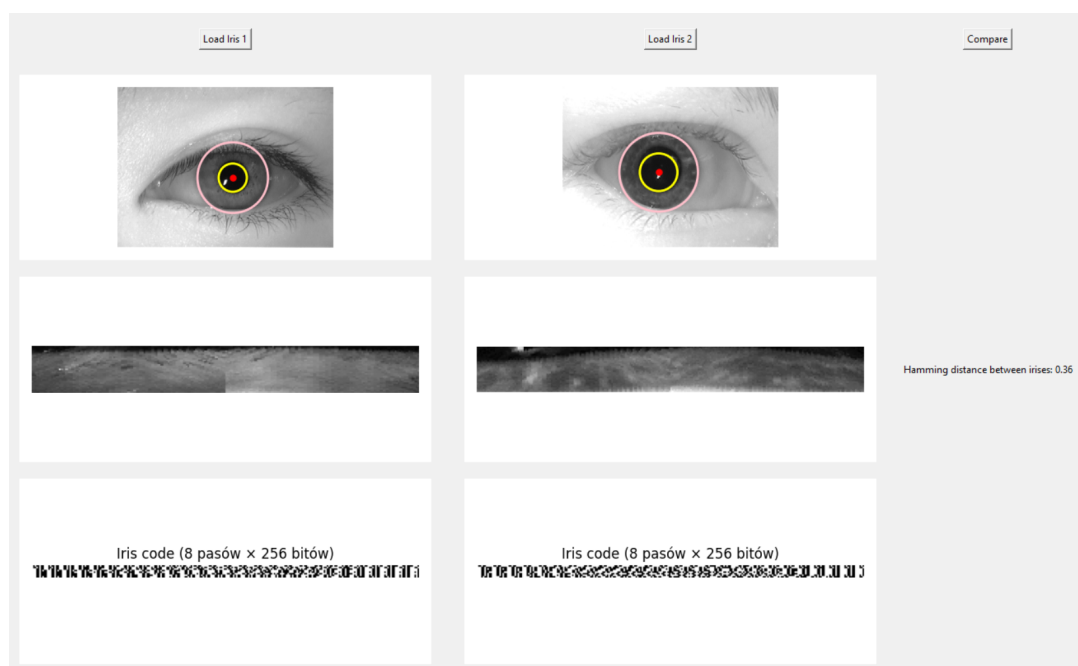
Rysunek 2: Dwa obrazy przedstawiające tę samą tęczówkę, wykonane w różnych warunkach (np. inne oświetlenie, inna pozycja źrenicy)

W powyższym przykładzie przedstawiono dwa obrazy pochodzące z tego samego oka. Pomimo możliwych różnic w położeniu źrenicy czy oświetleniu, kodowanie tęczówki za pomocą cech Gaborowskich umożliwia skuteczne rozpoznanie podobieństwa. Obliczona odległość Hamminga pomiędzy wygenerowanymi kodami binarnymi wyniosła 0.28. Taka wartość oznacza wysoki poziom zgodności, co wskazuje, że obrazy reprezentują tę samą osobę.

Na rysunku ?? widoczne są dwa obrazy przedstawiające różne tęczówki – pobrane od dwóch różnych osób. Zastosowany algorytm kodowania tęczówki generuje dla każdego obrazu unikalny wzorec binarny. Porównanie tych wzorców przy pomocy metryki Hamminga dało wynik równy 0.40, co sugeruje wyraźną różnicę pomiędzy kodami. Taka wartość wskazuje, że tęczówki nie pochodzą od tej samej osoby, zgodnie z oczekiwaniami.

4 Podsumowanie

Celem projektu było opracowanie aplikacji umożliwiającej rozpoznawanie osoby na podstawie analizy obrazu tęczówki oka. Zbudowane narzędzie realizuje kompletny proces przetwarzania



Rysunek 3: Dwa obrazy przedstawiające różne tęczówki – pochodzące od dwóch różnych osób obrazu – od jego wczytania, przez konwersję do odcieni szarości, segmentację źrenicy i tęczówki, normalizację obszaru interesującego, aż po ekstrakcję cech i kodowanie wzorca w postaci binarnej. Porównanie kodów binarnych odbywa się przy użyciu metryki Hamminga, co pozwala ocenić stopień podobieństwa pomiędzy dwiema tęczówkami.

Testy przeprowadzone na parach obrazów tej samej oraz różnych tęczówek potwierdziły skuteczność metody. Dla obrazów tego samego oka uzyskano niską wartość odległości Hamminga (0.28), natomiast dla obrazów różnych osób – wartość wyraźnie wyższą (0.40). Otrzymane wyniki wskazują, że opracowane rozwiązanie może być wykorzystywane jako narzędzie wspomagające systemy biometryczne.

Zastosowanie bibliotek *Pillow*, *numpy* oraz *matplotlib*, a także środowiska *Tkinter* pozwoliło na stworzenie funkcjonalnej aplikacji z graficznym interfejsem użytkownika. Użyte metody przetwarzania obrazu są stosunkowo proste, a jednocześnie skuteczne, co sprawia, że aplikacja może być podstawą do dalszej rozbudowy – np. o automatyczne dopasowanie parametrów progowania, dokładniejsze metody segmentacji czy integrację z bazami danych biometrycznych.